

POCKET SMART AI

AI-Powered Personal Finance Web Application

PROJECT REPORT

Project Focus	Personal finance assistance, expense awareness and AI-supported interaction
Backend	Python • FastAPI • Uvicorn
Data Layer	SQLite database
AI Layer	Gemini AI utility integration
Frontend	HTML / CSS templates and static resources

Prepared for Project Submission

01 | PROJECT OVERVIEW

Pocket Smart AI is a web-based personal finance assistant that combines a Python backend, a lightweight database and AI-supported functionality in one application. The project is designed to make financial information easier to work with and to provide a foundation for intelligent assistance.

Abstract

The application demonstrates an end-to-end software workflow: a browser-based interface communicates with a FastAPI backend; the backend works with stored application data; and Gemini-related utilities support AI functionality. The project is structured into reusable files and folders so that backend, database, AI utilities and user-interface resources can be maintained separately.

Project Vision

“Make everyday financial information easier to understand, manage and act on through a simple AI-assisted experience.”

Key Project Characteristics

- Modular Python project structure with a central FastAPI application.
- Local SQLite database for lightweight and accessible data storage.
- Gemini AI utility layer for AI-assisted functionality.
- Separate templates and static resources for the web interface.
- Local development and testing through Uvicorn with reload support.

02 | PROBLEM, NEED & OBJECTIVES

Problem Statement

Many people find it difficult to keep track of daily spending and understand their financial habits consistently. Information may be recorded in different places, making it harder to identify patterns or obtain useful guidance. A simple digital assistant can bring these activities into a single workflow.

Need for the System

Challenge	Project Response
Scattered financial information	Centralized application workflow
Manual interpretation of spending	AI-assisted interaction and insights
Complex application setup	Lightweight FastAPI + SQLite architecture
Difficulty extending features	Modular files and reusable components

Objectives

- Develop a working web application for personal finance assistance.
- Provide a backend using Python and FastAPI.
- Store project data using SQLite.
- Integrate Gemini AI utilities into the application workflow.
- Create a clear interface using templates and static resources.
- Test the application locally and document the completed workflow.

Expected Outcome

The expected outcome is a working prototype that can be launched locally, accessed through a browser and extended with additional finance-oriented features in future versions.

03 | TECHNOLOGY STACK & ARCHITECTURE

Layer	Technology / File	Role
Application	Python	Core development language
Backend	FastAPI	Routes, requests and application logic
Server	Uvicorn	Runs the FastAPI application locally
AI	gemini_utils.py	AI-related helper functionality
Database	pocketsmart.db	Local structured data storage
UI	templates / static	Pages, styling and frontend resources

High-Level Architecture

USER	WEB INTERFACE	FASTAPI BACKEND	DATA + AI
Browser interaction	Templates + Static	main.py / routes	SQLite + Gemini utilities

Architecture Flow

Browser → Web Interface → FastAPI Request Handling → Application Logic → SQLite / Gemini Utility →
Response to Browser

The architecture keeps the presentation, backend logic, data storage and AI utilities separated. This makes the project easier to test, maintain and extend.

04 | MODULES & PROJECT FILE STRUCTURE

Core Modules

main.py

Central FastAPI application. It coordinates web routes, application behavior and the connection between the user interface and backend logic.

gemini_utils.py

Utility layer for Gemini AI-related operations. Keeping AI helpers separate improves modularity.

pocketsmart.db

SQLite database file used for local application data storage.

templates/

Contains web page templates used to render the application's interface.

static/

Contains frontend static resources such as styles and other supporting assets.

requirements.txt

Lists the Python packages required for the project environment.

Observed Project Structure

```
pocket smart AI/  
  main.py  
  gemini_utils.py  
  pocketsmart.db  
  requirements.txt  
  templates/  
  static/  
  __pycache__/
```

Modular Design Benefit

Separating the backend, AI utility, database, templates and static resources creates a cleaner development structure. Each area can be improved without requiring the entire application to be rewritten.

05 | IMPLEMENTATION WORKFLOW

Environment Setup

Create the project folder and install the required Python packages.

Backend Development

Build the FastAPI application in main.py and define the required application behavior.

AI Integration

Use gemini_utils.py to keep Gemini-related functionality organized.

Data Storage

Use pocketsmart.db as the local SQLite data store.

Frontend Integration

Connect templates and static resources to the backend application.

Local Execution

Launch the application through Uvicorn and verify the server startup.

Browser Verification

Open the local application in a browser and verify the working interface.

Execution Command

```
python -m uvicorn main:app --reload
```

The local server output shown during development indicates that the application started successfully and reached the application startup stage.

06 | PROJECT WORKFLOW EVIDENCE

Development Workflow

The project workspace is organized into major development stages. The visible workflow includes Gemini AI initialization, core functionality development, FastAPI backend integration, UI development, testing/deployment and conclusion.

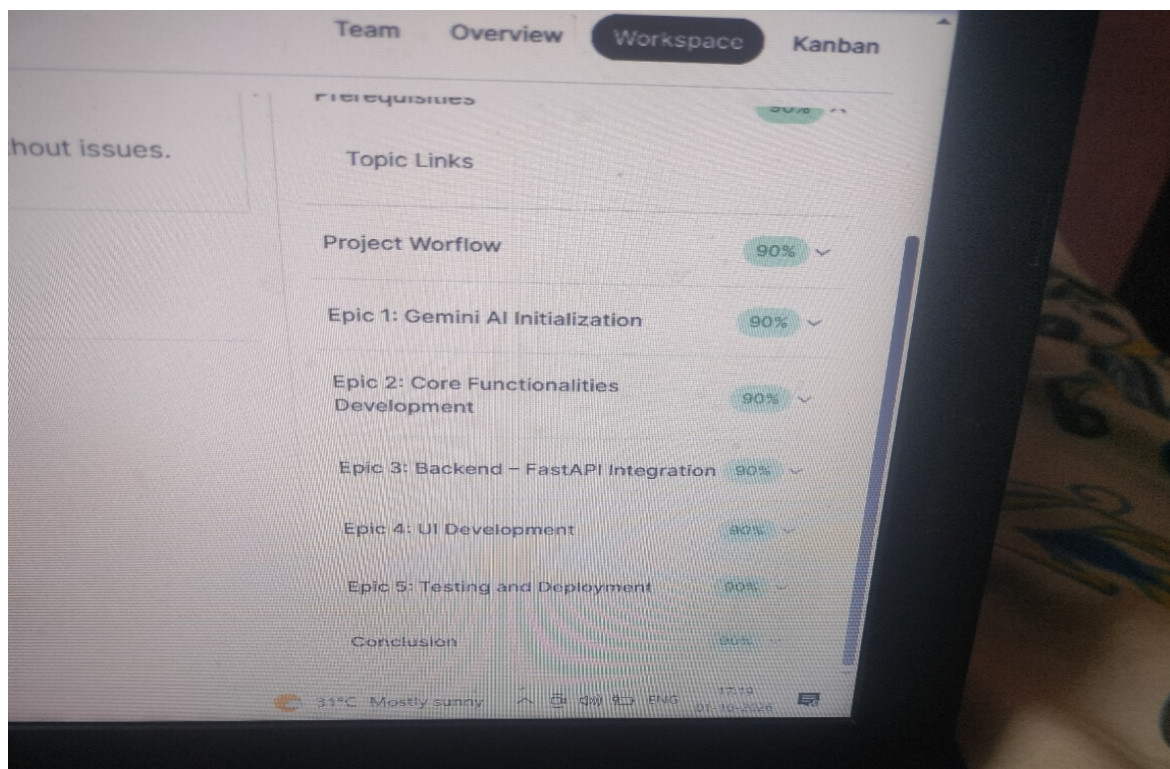


Figure 1. Project workflow showing the major development epics.

Workflow Interpretation

- **Epic 1 — Gemini AI Initialization:** establishes the AI-related foundation.
- **Epic 2 — Core Functionalities Development:** focuses on the main application capabilities.
- **Epic 3 — Backend – FastAPI Integration:** connects the application logic to the web backend.
- **Epic 4 — UI Development:** develops the user-facing web experience.
- **Epic 5 — Testing and Deployment:** verifies the application and prepares it for use.

07 | BACKEND EXECUTION EVIDENCE

Local Server Startup

The development environment shown below contains the main application files and the command used to launch the FastAPI application through Uvicorn. The terminal output indicates that the server started, a reloader process was created and the application startup completed.

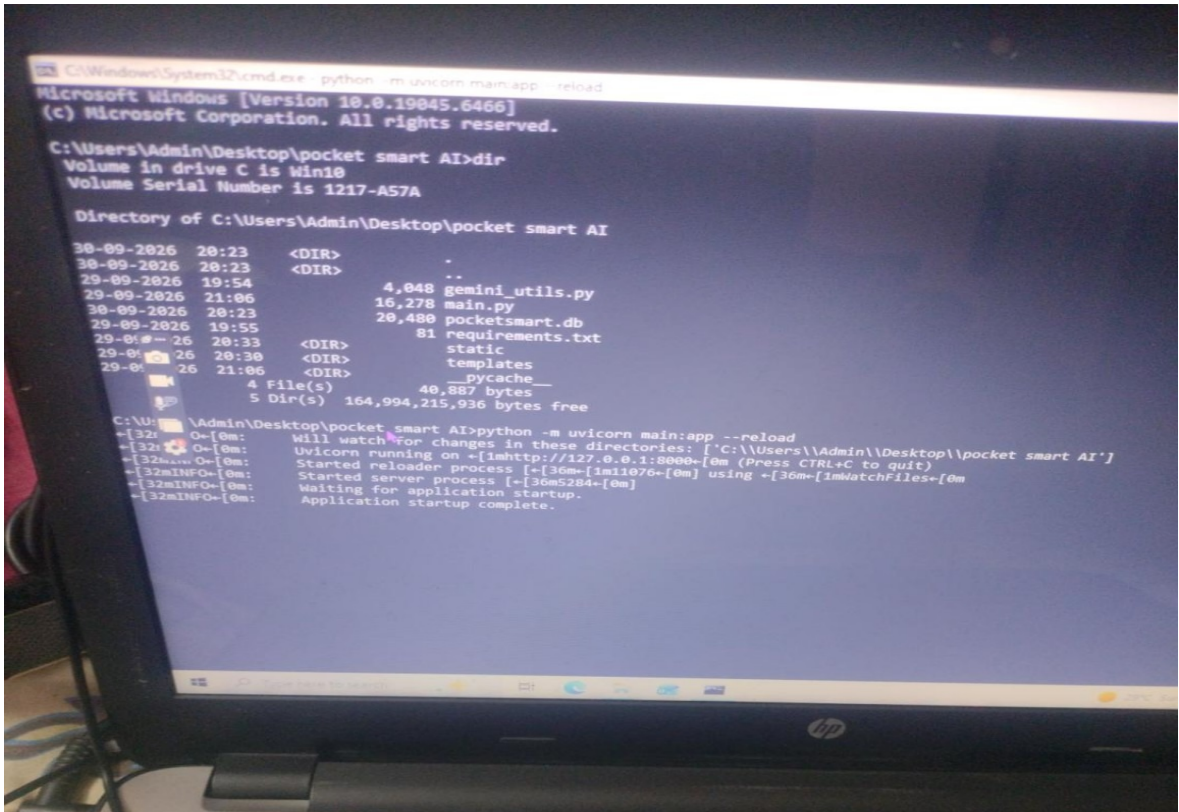


Figure 2. Local project directory and Uvicorn/FastAPI startup output.

Evidence Summary

Evidence	What it demonstrates
main.py	FastAPI application entry point is present.
gemini_utils.py	AI utility module is present.
pocketsmart.db	Local database file is present.
requirements.txt	Project dependency list is present.
Uvicorn startup	Backend application starts successfully in the local environment.

08 | TESTING, RESULTS & QUALITY CHECKS

Testing Approach

Testing was approached in stages: verify the project files, start the backend server, confirm application startup, open the application through the browser and inspect the user-facing interface. This process helps identify setup, backend and interface issues before submission.

Test Area	Check	Expected Result
Project files	Required files/folders available	Application structure is complete
Server startup	Run Uvicorn command	Server starts without startup failure
Application startup	Observe server logs	Startup completes successfully
Browser access	Open local application	Web interface loads
UI verification	Inspect visible pages/features	Interface behaves as designed

Observed Result

The provided development evidence shows the project directory, required application files and successful FastAPI/Uvicorn startup. The project workflow also shows dedicated stages for AI initialization, core functionality, backend integration, UI development and testing/deployment.

Quality Considerations

- Keep API keys and sensitive configuration outside source code when deploying.
- Validate user inputs before storing or processing them.
- Use clear error handling for database and AI-service failures.
- Test important user flows after every major feature change.
- Back up or migrate the database appropriately when moving beyond local development.

09 | FUTURE ENHANCEMENTS & CONCLUSION

Future Enhancements

User Authentication: Add secure user accounts and individualized financial data.

Expense Analytics: Introduce charts and summaries for spending categories and time periods.

Budget Planning: Allow users to define budgets and monitor progress against them.

Smart Alerts: Provide configurable reminders and notifications for financial activities.

Advanced AI Assistance: Expand AI prompts and contextual insights while applying appropriate privacy controls.

Deployment: Move from local development to a secure hosted environment with production configuration.

Conclusion

Pocket Smart AI demonstrates how modern web development and generative AI can be combined to create a practical personal finance assistant. The project uses Python and FastAPI for backend development, SQLite for lightweight data storage, Gemini-related utilities for AI functionality, and templates/static resources for the user interface.

The completed development workflow and local execution evidence provide a foundation for further testing, refinement and deployment. With additional security, analytics, authentication and user-experience improvements, the application can be extended into a more complete personal finance platform.

POCKET SMART AI

AI • WEB • FINANCE

End of Project Report